

Lab Assignment -- 4

Implementation of a Concurrent Network Information Service. Implement a client-server application in C in which a concurrent TCP server provides DNS-like lookup services to multiple clients simultaneously. The application should also use UDP for a simple server-status query.

Problem 1

Implementation of a Concurrent TCP Lookup Server

The server maintains a small set of predefined DNS-like records in memory. Each record may contain:

- Hostname
- Record type: A, CNAME, MX, or NS
- Corresponding value

Example records:

```
www.example.com A 192.168.1.10
mail.example.com A 192.168.1.20
web.example.com CNAME www.example.com
example.com MX mail.example.com
example.com NS ns1.example.com
```

The client should allow the user to enter:

1. A host or a domain name
2. A record type

The server searches its records and returns the corresponding value. If no matching record exists, it should return "Record not found".

The TCP server must support multiple clients concurrently using threads. The server creates one thread for every connected client. It should also allow each client to perform multiple queries before disconnecting. The server should be able to display the IP address and port number of each connected client.

The client should terminate when the user enters EXIT.

Problem 2

Implementing a UDP Server Status Service

Using the same server program or a separate UDP server, provide a simple status service on another port.

A client sends:

STATUS

using UDP.

The server replies with information such as:

Server active

and the number of TCP clients currently connected.

Sample interaction:

Client 1:

Enter domain: www.example.com

Enter type: A

Server response:

192.168.1.10

Client 2:

Enter domain: example.com

Enter type: MX

Server response:

mail.example.com

UDP status query:

STATUS

Response:

Server active. Connected clients: 3

Extension Homework (No submission required)

Use Wireshark capture to do the following:

1. Observe TCP connection establishment. Start packet capture, connect one client to the concurrent server, and identify the three packets of the TCP three-way handshake.

Record the client/server IP addresses, client ephemeral port, server port, and SYN/SYN-ACK/ACK sequence.

2. Compare multiple concurrent clients. Connect at least three clients simultaneously. Apply an appropriate Wireshark filter and identify the separate TCP connections. Observe the four-tuple (Source IP, Source Port, Destination IP, Destination Port) for each connection and try to explain how the server can distinguish clients even though all connect to the same server port.
3. Observe application data over TCP. Perform one DNS-like query such as `www.example.com`. A. Locate the TCP packets carrying the request and response and use Follow → TCP Stream to inspect the application-level exchange. Observe how the application protocol is carried as data inside TCP.
4. Compare TCP with UDP. Send the STATUS request to the UDP service and capture it. Identify the UDP request and response, and compare this exchange with the TCP lookup service. Students should specifically note that there is no connection establishment/three-way handshake for UDP.
5. From the Wireshark capture, try to find two differences that you observe between the TCP lookup service and the UDP status service.